Universidad de La Sabana

Aproximación de funciones

Reinforcement Learning, segundo semestre

Ignacio Brevis

Versión 1.0. Si encuentras algún error o tienes comentarios, escríbeme a `ignaciobrve@unisabana.edu.co`.

Resumen

Todos los algoritmos vistos hasta aquí guardan una estimación (un número) para cada estado o para cada par estado-acción. Cuando el espacio de estados es «grande», esa tabla deja de caber en memoria y, sobre todo, deja de poder llenarse. Esta guía estudia el caso en que la tabla se sustituye por una **función parametrizada**. Esta guía cubre, los casos en que las funciones de valor se aproximan por funciones (función de valor de estado y función de valor de acción) y cuando la política se aproxima por funciones. Se incluyen listados en Python de las partes algorítmicas, y al final 8 ejercicios con pistas y soluciones completas.

Índice

1. Por qué las tablas no bastan **2**
- 1.1. El problema . . . 2
- 1.2. Notación . . . 2
- 1.3. El entorno de los ejemplos . . . 3

Parte I: Métodos de función de valor **3**

2. El objetivo y cómo minimizarlo **3**
- 2.1. La función objetivo . . . 3
- 2.2. Descenso de gradiente . . . 4
- 2.3. Sustituir el objetivo desconocido . . . 4
- 2.4. El caso lineal . . . 4
- 2.5. Un ejemplo: aproximar v_* del gridworld . . . 5

3. Valores de acción y Deep Q-learning **6**
- 3.1. Sarsa y Q-learning con aproximación . . . 6
- 3.2. Deep Q-learning . . . 6

Parte II: Gradiente de política **7**

4. Políticas parametrizadas **7**
- 4.1. Representación . . . 7
- 4.2. Qué se optimiza . . . 7
- 4.3. El teorema del gradiente de política . . . 8
- 4.4. REINFORCE . . . 8

Parte III: Actor-critic **8**

5. Actor-critic **8**
5.1. QAC: el más simple 9
5.2. A2C: la ventaja 9
5.3. Actor-critic off-policy 10
5.4. Actor-critic determinista 10

6. Visión unificada **10**

7. Resumen **11**

8. Ejercicios **11**

9. Soluciones **13**

---

1. Por qué las tablas no bastan

La idea en una frase. Una tabla trata cada estado como si no tuviera nada que ver con los demás. Una función parametrizada permite que lo aprendido en un estado sirva también para sus vecinos.

1.1. El problema

Representar v_π mediante una tabla significa almacenar $|\mathcal{S}|$ números independientes, y q_π significa almacenar $|\mathcal{S}||\mathcal{A}|$. Eso plantea dos dificultades distintas, y conviene separarlas porque sólo una de ellas es de memoria:

- **Almacenamiento**: con 10^{10} estados la tabla no cabe. Es un problema real pero, en cierto sentido, el menos interesante.
- **Generalización**: aunque cupiera, para estimar la entrada de un estado hay que *visitarlo*. Una tabla no extrae ninguna conclusión sobre un estado a partir de lo observado en otro, por parecidos que sean. Con 10^{10} estados, la inmensa mayoría nunca se visitará ni una vez.

La segunda dificultad es la decisiva. La solución consiste en describir el valor mediante una función con muchos menos parámetros que estados,

$$\hat{v}(s, w) \approx v_\pi(s), \qquad w \in \mathbb{R}^m \text{ con } m \ll |\mathcal{S}|,$$

de modo que al ajustar w con las muestras de un estado cambie también el valor estimado de todos los estados que compartan características con él.

Cuidado. La generalización tiene un precio inevitable: si $m < |\mathcal{S}|$, en general **no** existe ningún w con $\hat{v}(s, w) = v_\pi(s)$ para todo s. Pasamos de calcular el valor exacto a buscar la mejor aproximación dentro de una familia restringida. Todo lo que sigue consiste en precisar qué significa «la mejor».

1.2. Notación

Tener en cuenta que hay **dos** vectores de parámetros con papeles distintos: w parametriza valores y θ parametriza políticas.

Símbolo	Significa
$\hat{v}(s,w)$, $\hat{q}(s,a,w)$	valor de estado y de acción aproximados
$w \in \mathbb{R}^m$	vector de parámetros del *valor*
θ	vector de parámetros de la *política*
$\phi(s)$, $\phi(s,a)$	vector de características
$\pi(a\|s;\theta)$	política parametrizada
$J(w)$, $J(\theta)$	función objetivo a minimizar o maximizar
$d_\pi(s)$	distribución estacionaria de los estados bajo π
α_w, α_θ	tasas de aprendizaje del crítico y del actor
δ_t	error TD
∇_w, ∇_θ	gradiente respecto de cada vector

1.3. El entorno de los ejemplos

Los ejemplos numéricos usan la cuadrícula de 5×5: cinco acciones (arriba, derecha, abajo, izquierda, quedarse), movimiento determinista, $+1$ al entrar en la casilla azul, -1 al entrar en una naranja o al chocar con una pared, 0 en las blancas, y $\gamma = 0.9$.

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

Con 25 estados una tabla es perfectamente viable; lo usamos precisamente por eso, porque permite comparar la aproximación contra el valor exacto.

Parte I: Métodos de función de valor

2. El objetivo y cómo minimizarlo

La idea en una frase. Aproximar v_π es un problema de **mínimos cuadrados**: elegir w que minimice el error cuadrático medio entre $\hat{v}(s,w)$ y $v_\pi(s)$, pesando cada estado por lo a menudo que se visita.

2.1. La función objetivo

Definición 2.1 (Objetivo)**.**

$$J(w) \stackrel{\text{def}}{=} \mathbb{E}\big[(v_\pi(S) - \hat{v}(S,w))^2\big] = \sum_s d_\pi(s)\,\big(v_\pi(s) - \hat{v}(s,w)\big)^2. \tag{1}$$

Aquí d_π es la distribución de los estados. La elección natural es la **distribución estacionaria** de la cadena de Markov inducida por π: la fracción de tiempo que el agente pasa en cada estado a largo plazo, caracterizada por

$$d_\pi^\top P_\pi = d_\pi^\top, \qquad \textstyle\sum_s d_\pi(s) = 1.$$

Observación. El peso $d_\pi(s)$ no es un detalle técnico. Dice que **los errores en estados que el agente visita a menudo importan más** que los errores en estados que casi nunca pisa. Esto es razonable, pero tiene una consecuencia incómoda: el objetivo depende de la política, de modo que al cambiar la política cambia también la noción de «buena aproximación». Volveremos sobre esto al hablar de los métodos off-policy.

2.2. Descenso de gradiente

Minimizar (1) con descenso de gradiente requiere

$$\nabla_w J(w) = -2\,\mathbb{E}\big[(v_\pi(S) - \hat{v}(S,w))\,\nabla_w \hat{v}(S,w)\big],$$

y absorbiendo el 2 en la tasa de aprendizaje, el descenso de gradiente estocástico da

$$w_{t+1} = w_t + \alpha_w (v_\pi(s_t) - \hat{v}(s_t,w_t))\,\nabla_w \hat{v}(s_t,w_t). \tag{2}$$

Cuidado. La ecuación (2) **no se puede implementar**: contiene $v_\pi(s_t)$, que es justamente lo que no conocemos. Toda esta parte consiste en sustituir ese término por algo observable.

2.3. Sustituir el objetivo desconocido

Hay dos sustituciones naturales, y dan los dos grandes métodos sin modelo:

- **Monte Carlo**: reemplazar $v_\pi(s_t)$ por el retorno observado g_t, que es un estimador insesgado:
$$w_{t+1} = w_t + \alpha_w (g_t - \hat{v}(s_t,w_t))\nabla_w \hat{v}(s_t,w_t).$$
- **Diferencias temporales**: reemplazarlo por el objetivo bootstrap $r_{t+1} + \gamma \hat{v}(s_{t+1},w_t)$:
$$w_{t+1} = w_t + \alpha_w \big[\, r_{t+1} + \gamma \hat{v}(s_{t+1},w_t) - \hat{v}(s_t,w_t) \,\big]\nabla_w \hat{v}(s_t,w_t). \tag{3}$$

Cuidado (Por qué se llama *semi*gradiente). En (3) el objetivo $r_{t+1} + \gamma \hat{v}(s_{t+1},w_t)$ **también depende de** w. Si derivásemos J correctamente, aparecería además un término $-\gamma\,\nabla_w \hat{v}(s_{t+1},w)$, que la fórmula omite. Es decir, (3) no es exactamente el gradiente de la función objetivo (después de reemplazar $v_\pi(s_t)$). De ahí el nombre *semigradiente*.
Esto no es un descuido sino una decisión deliberada: el semigradiente converge más rápido y permite aprender en línea.

2.4. El caso lineal

La familia más estudiada es la aproximación lineal (lineal en los parámetros):

$$\hat{v}(s,w) = \phi^{\mathsf{T}}(s)\,w, \qquad \nabla_w \hat{v}(s,w) = \phi(s). \tag{4}$$

El gradiente es simplemente el vector de características, de modo que (3) se convierte en

$$w_{t+1} = w_t + \alpha_w \big[r_{t+1} + \gamma \phi^{\mathsf{T}}(s_{t+1}) w_t - \phi^{\mathsf{T}}(s_t) w_t\big]\phi(s_t).$$

Observación. El caso tabular es un caso particular del lineal: basta tomar como características los vectores canónicos, $\phi(s) = e_s$. Entonces $\hat{v}(s,w) = w_s$ y la actualización afecta a una sola componente, que es exactamente el TD tabular. Dicho de otro modo, la aproximación lineal no sustituye a la tabla sino que la contiene como caso extremo, el de máxima capacidad y mínima generalización.

2.5. Un ejemplo: aproximar v_* del gridworld

Tomemos el valor óptimo v_* de la cuadrícula de la Sección 1.3, que como tabla son 25 números:

5.83	5.58	6.20	6.48	5.83
6.48	7.20	8.00	7.20	6.48
7.20	8.00	10.00	8.00	7.20
8.00	10.00	10.00	10.00	8.00
7.20	9.00	10.00	9.00	8.10

Si x e y denotan fila y columna, podemos intentar describir esos 25 números con unos pocos parámetros. Con un plano, $\phi(s) = [1,\, x,\, y]^\mathsf{T}$, son tres parámetros; añadiendo los términos cuadráticos $\phi(s) = [1,\, x,\, y,\, x^2,\, xy,\, y^2]^\mathsf{T}$, son seis. Ajustando por mínimos cuadrados:

Características	**Parámetros**	**RMSE**
$[1,\, x,\, y]$	3	0.9278
$[1,\, x,\, y,\, x^2,\, xy,\, y^2]$	6	0.5135
tabular ($\phi(s) = e_s$)	25	0

Observación. Estos tres ejemplos muestran el compromiso completo en miniatura. Con tres parámetros el error es apreciable, pero hemos comprimido 25 números en 3 y cualquier estado nuevo recibe una estimación razonable sin haber sido visitado. Con 25 parámetros el error es cero y no hay generalización ninguna. El salto de 3 a 6 reduce el error casi a la mitad, lo que indica que la superficie verdadera tiene curvatura: v_* crece hacia el objetivo y luego se aplana, y un plano no puede capturar eso.

Listado 1: Ajuste por mínimos cuadrados de v_* con características polinómicas.

```python
import numpy as np

# rows[s], cols[s]: fila y columna del estado s;  v: v* tabular (25 valores)
rows = np.array([s // 5 for s in range(25)], float)
cols = np.array([s %  5 for s in range(25)], float)

def fit(PHI, v):
    """Mejor w en el sentido de minimos cuadrados, y el error resultante."""
    w, *_ = np.linalg.lstsq(PHI, v, rcond=None)
    v_hat = PHI @ w
    return w, v_hat, np.sqrt(np.mean((v - v_hat) ** 2))

PHI_lin  = np.c_[np.ones(25), rows, cols]                                  # 3 parametros
PHI_quad = np.c_[np.ones(25), rows, cols, rows**2, rows*cols, cols**2]     # 6 parametros

for name, PHI in [("plano", PHI_lin), ("cuadratica", PHI_quad)]:
    w, v_hat, rmse = fit(PHI, v)
    print(f"{name:12s} m={PHI.shape[1]:2d}  RMSE={rmse:.4f}")
```

Cuidado. Este ajuste por mínimos cuadrados **no es** lo que hace un agente de RL: aquí hemos usado v_*, que el agente no conoce. Sirve únicamente para medir *cuál es el mejor resultado alcanzable* con cada familia de características. Un agente que aprende con (3) no puede hacerlo mejor que eso, y en general se queda algo por debajo.

3. Valores de acción y Deep Q-learning

3.1. Sarsa y Q-learning con aproximación

Para mejorar la política hace falta q, no v. Las dos reglas se obtienen sustituyendo $\hat{v}$ por $\hat{q}$ en (3):

$$\text{Sarsa:} \quad w_{t+1} = w_t + \alpha_w \big[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t)\big] \nabla_w \hat{q}(s_t, a_t, w_t), \tag{5}$$

$$\text{Q-learning:} \quad w_{t+1} = w_t + \alpha_w \big[r_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t)\big] \nabla_w \hat{q}(s_t, a_t, w_t). \tag{6}$$

La estructura es la misma de siempre: *parámetros nuevos = parámetros viejos + paso × error TD × gradiente*. Lo único que cambia respecto del caso tabular es que el error TD ya no se asigna a una celda, sino que se reparte entre los m parámetros en proporción al gradiente.

3.2. Deep Q-learning

Deep Q-learning representa $\hat{q}(s, a, w)$ con una red neuronal. El objetivo que minimiza es el error cuadrático de la ecuación de Bellman de optimalidad:

$$J(w) = \mathbb{E}\Big[\big(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\big)^2\Big]. \tag{7}$$

Aquí aparece un problema técnico que explica toda la arquitectura del algoritmo: w figura *dos veces* dentro del paréntesis, y derivar respecto de ambas es incómodo. La solución de Deep Q-learning consiste en congelar una de ellas.

Definición 3.1 (Las dos redes)**.** Se mantienen dos juegos de parámetros:

- la **red principal**, con parámetros w, que se actualiza en cada iteración;
- la **red objetivo**, con parámetros w_T, que permanece *fija* y se copia de la principal cada cierto número de iteraciones.

El objetivo $y = r + \gamma \max_a \hat{q}(s', a, w_T)$ se calcula con la red objetivo y se trata como una constante al derivar, de modo que

$$\nabla_w J \;\approx\; -2\,\mathbb{E}\big[(y - \hat{q}(S, A, w))\, \nabla_w \hat{q}(S, A, w)\big].$$

Definición 3.2 (Repetición de experiencia)**.** Las transiciones (s, a, r, s') observadas se almacenan en un *buffer*, y las actualizaciones se calculan sobre elementos extraídos **uniformemente al azar** de él, no sobre la transición más reciente.

Observación (Por qué hace falta el buffer)*.* Hay dos razones, y conviene distinguirlas. La primera es estadística: el objetivo (7) es una esperanza sobre una distribución que se supone uniforme sobre los pares (s, a), y las transiciones consecutivas de una trayectoria están fuertemente correlacionadas, de modo que muestrearlas en orden no estima esa esperanza. Extraer al azar del buffer rompe la correlación. La segunda es de eficiencia: cada transición puede reutilizarse muchas veces en lugar de descartarse tras un solo uso.

Notemos que esto es legítimo porque Q-learning es **off-policy**: su objetivo no depende de la política que generó los datos. generadas por una política antigua sería incorrecto.

Algoritmo 1 (Deep Q-learning)**. Inicializar:** red principal w; red objetivo $w_T = w$; buffer vacío.

Para cada paso:

1. ejecutar una acción según una política ε-greedy respecto de $\hat{q}(\cdot,\cdot,w)$ y guardar (s,a,r,s') en el buffer;
2. extraer del buffer un mini-batch uniforme de transiciones;
3. para cada una, calcular el objetivo $y = r + \gamma \max_{a'} \hat{q}(s',a',w_T)$ con la *red objetivo*;
4. actualizar w minimizando $\sum (y - \hat{q}(s,a,w))^2$ sobre el lote;
5. cada C pasos, copiar $w_T \leftarrow w$.

Cuidado. La red objetivo no puede quedar congelada para siempre: si w_T nunca se actualiza, el algoritmo converge a la solución de un problema que no es el que queríamos resolver. El periodo C es un compromiso entre estabilidad (que w_T cambie poco) y corrección (que w_T acabe reflejando lo aprendido).

Parte II: Gradiente de política

4. Políticas parametrizadas

La idea en una frase. La misma idea de sustituir una tabla por una función, aplicada ahora a la *política*. En lugar de deducir la política de la funciones de valor, se la optimiza directamente mediante un ascenso de gradiente.

4.1. Representación

Hasta ahora la política era una tabla de probabilidades $\pi(a|s)$. Ahora es una función $\pi(a|s;\theta)$. La parametrización más común es la **softmax** sobre unas preferencias $h(s,a;\theta)$:

$$\pi(a|s;\theta) = \frac{e^{h(s,a;\theta)}}{\sum_{a'} e^{h(s,a';\theta)}}. \tag{8}$$

Observación. La softmax garantiza automáticamente que las probabilidades son positivas y suman 1, sea cual sea θ: no hay que imponer ninguna restricción al optimizar. Además la política resultante es **estocástica con probabilidad positiva en todas las acciones**, de modo que la exploración queda incorporada en la propia representación y no hace falta añadir un mecanismo ε-greedy por fuera.

4.2. Qué se optimiza

Hace falta una cantidad escalar que mida lo buena que es una política. Aquí consideramos dos:

- el **valor medio de estado** $\bar{v}_\pi = \sum_s d(s)\, v_\pi(s)$;
- la **recompensa media** $\bar{r}_\pi = \sum_s d_\pi(s) r_\pi(s) = \sum_s d_\pi(s) \sum_a \pi(a|s;\theta)\, r(s,a)$.

Ambas son funciones de θ, y el problema pasa a ser $\max_\theta J(\theta)$ resuelto por **ascenso** de gradiente, $\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta J(\theta_t)$.

4.3. El teorema del gradiente de política

El obstáculo es que $J(\theta)$ depende de θ de dos maneras: a través de las probabilidades $\pi(a|s;\theta)$ y a través de la distribución de estados d_π, que también cambia al cambiar la política. El resultado central dice que, pese a ello, el gradiente tiene una forma sorprendentemente simple en la que la derivada de d_π no aparece.

Teorema 4.1 (Gradiente de política)**.**

$$\nabla_\theta J(\theta) = \sum_s \eta(s) \sum_a \nabla_\theta \pi(a|s;\theta)\, q_\pi(s,a) = \mathbb{E}_{S\sim\eta,\, A\sim\pi}\big[\nabla_\theta \ln \pi(A|S;\theta)\, q_\pi(S,A)\big], \tag{9}$$

donde η es una distribución de estados.

La segunda igualdad es la que hace el teorema utilizable, y se obtiene del *truco del logaritmo*:

$$\nabla_\theta \ln \pi(a|s;\theta) = \frac{\nabla_\theta \pi(a|s;\theta)}{\pi(a|s;\theta)} \implies \nabla_\theta \pi(a|s;\theta) = \pi(a|s;\theta)\, \nabla_\theta \ln \pi(a|s;\theta),$$

lo que convierte la suma sobre acciones en una esperanza respecto de la propia política, y por tanto en algo estimable muestreando.

Cuidado. Como $\ln \pi$ aparece en (9), hace falta $\pi(a|s;\theta) > 0$ para toda acción. Es otra razón para la softmax, que lo garantiza por construcción. Una política determinista no funciona, y por eso el caso determinista necesita un teorema aparte (Sección 5.4).

4.4. REINFORCE

Sustituyendo la esperanza de (9) por una sola muestra se obtiene el ascenso de gradiente estocástico

$$\theta_{t+1} = \theta_t + \alpha_\theta\, \nabla_\theta \ln \pi(a_t|s_t;\theta_t)\, q_t(s_t,a_t), \tag{10}$$

donde q_t es una estimación de $q_\pi(s_t,a_t)$. Si esa estimación se obtiene por **Monte Carlo** (el retorno observado del episodio), el algoritmo se llama **REINFORCE**.

Observación (Qué hace la fórmula)*.* Conviene reescribir (10) usando el truco del logaritmo al revés:

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{q_t(s_t,a_t)}{\pi(a_t|s_t;\theta_t)} \nabla_\theta \pi(a_t|s_t;\theta_t).$$

Leída así, dice que la probabilidad de la acción tomada sube si $q_t > 0$ y baja si $q_t < 0$, con un paso proporcional a q_t y *inversamente* proporcional a la probabilidad que ya tenía. Este último factor compensa el muestreo: las acciones raras, cuando aparecen, pesan más, porque aparecerán pocas veces.

Observación. Que las componentes del gradiente sumen cero no es casual: subir la probabilidad de una acción obliga a bajar la de las demás, porque la softmax las fuerza a sumar 1. Esa propiedad es la que hace que la invarianza al *baseline* de la sección siguiente funcione.

Parte III: Actor-critic

5. Actor-critic

La idea en una frase. En (10) hace falta estimar $q_\pi(s_t,a_t)$. Si esa estimación se hace con TD en lugar de Monte Carlo, el algoritmo pasa a llamarse actor-critic: el *actor* actualiza la

política, el *crítico* estima los valores.

5.1. QAC: el más simple

Algoritmo 2 (QAC)**.** En cada paso t, generando $a_t \sim \pi(\cdot|s_t;\theta_t)$ y observando $r_{t+1}, s_{t+1}, a_{t+1}$:

$$\begin{aligned} \text{Actor:} \quad & \theta_{t+1} = \theta_t + \alpha_\theta \, \nabla_\theta \ln \pi(a_t|s_t;\theta_t) \, \hat{q}(s_t, a_t, w_t), \\ \text{Crítico:} \quad & w_{t+1} = w_t + \alpha_w \big[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \big] \nabla_w \hat{q}(s_t, a_t, w_t). \end{aligned}$$

El crítico es exactamente Sarsa con aproximación, ecuación (5). La diferencia con REINFORCE es cómo se calcula q_t: Monte Carlo o TD. Como consecuencia, QAC aprende *en línea*, sin esperar al final del episodio.

5.2. A2C: la ventaja

Propiedad 5.1 (Invarianza al baseline)**.** *Para cualquier función $b(s)$ que no dependa de la acción,*

$$\mathbb{E}\big[\nabla_\theta \ln \pi(A|S;\theta) \, q_\pi(S,A)\big] = \mathbb{E}\big[\nabla_\theta \ln \pi(A|S;\theta) \, (q_\pi(S,A) - b(S))\big].$$

Demostración. Basta ver que el término añadido se anula. Para cada estado s,

$$\sum_a \pi(a|s;\theta) \, \nabla_\theta \ln \pi(a|s;\theta) \, b(s) = b(s) \sum_a \nabla_\theta \pi(a|s;\theta) = b(s) \, \nabla_\theta \underbrace{\sum_a \pi(a|s;\theta)}_{=1} = 0,$$

donde se ha usado el truco del logaritmo y que la derivada de una constante es nula. □

Observación. La invarianza dice que restar $b(s)$ **no cambia el gradiente verdadero**, pero sí cambia la *varianza* del estimador que lo aproxima con muestras. Por lo tanto, existe una familia entera de estimadores insesgados y podemos quedarnos con el de «menor»varianza. La elección estándar es $b(s) = v_\pi(s)$, que da la **función de ventaja** (advantage function)

$$A_\pi(s,a) \stackrel{\text{def}}{=} q_\pi(s,a) - v_\pi(s),$$

cuya lectura es directa: *cuánto mejor es la acción a que la media de lo que haría la política en ese estado.* Con ella, el actor sube la probabilidad de las acciones mejores que la media y baja la de las peores, en lugar de subir la de todas las acciones con retorno positivo.

El último paso es notar que la ventaja se estima con el error TD, porque

$$\mathbb{E}\big[R_{t+1} + \gamma v_\pi(S_{t+1}) - v_\pi(S_t) \;\big|\; S_t = s, A_t = a\big] = q_\pi(s,a) - v_\pi(s) = A_\pi(s,a).$$

El error TD es, en esperanza, exactamente la ventaja. Eso permite que el crítico estime sólo v en lugar de q.

Algoritmo 3 (Advantage actor-critic, A2C)**.**

$$\begin{aligned} \text{Error TD:} \quad & \delta_t = r_{t+1} + \gamma \, \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t), \\ \text{Actor:} \quad & \theta_{t+1} = \theta_t + \alpha_\theta \, \delta_t \, \nabla_\theta \ln \pi(a_t|s_t;\theta_t), \\ \text{Crítico:} \quad & w_{t+1} = w_t + \alpha_w \, \delta_t \, \nabla_w \hat{v}(s_t, w_t). \end{aligned}$$

Observación. El mismo número δ_t aparece en las actualizaciones del actor y del crítico. Esa coincidencia resume la idea del método: una única señal de sorpresa —la transición fue mejor o peor de lo esperado— corrige a la vez la estimación del valor y la probabilidad de la acción que se tomó.

5.3. Actor-critic off-policy

Si los datos los genera una política de *behaviour* $\beta \neq \pi$, el teorema del gradiente de política deja de aplicarse directamente, porque la esperanza de (9) se calcula con respecto a π. La corrección es el **muestreo por importancia** (importance sampling):

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\rho,\, A\sim\beta}\left[\frac{\pi(A|S;\theta)}{\beta(A|S)}\,\nabla_\theta \ln \pi(A|S;\theta)\, q_\pi(S,A)\right].$$

El algoritmo resultante es idéntico a A2C salvo por el factor de importancia

$$\frac{\pi(a_t|s_t;\theta_t)}{\beta(a_t|s_t)},$$

que multiplica tanto la actualización del actor como la del crítico.

Cuidado. El peso de importancia es una fuente notoria de varianza: si β y π se separan, el cociente puede hacerse enorme para algunas acciones y casi nulo para otras, y el estimador deja de ser utilizable en la práctica aunque siga siendo insesgado.

5.4. Actor-critic determinista

Todo lo anterior supone π estocástica, porque $\ln \pi$ aparece en el gradiente. Con una política determinista $a = \mu(s;\theta)$ hace falta un teorema distinto.

Teorema 5.2 (Gradiente de política determinista)**.**

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\rho}\Big[\nabla_\theta \mu(S;\theta)\,\left(\nabla_a q_\pi(S,a)\right)\big|_{a=\mu(S)}\Big].$$

Observación. En esta expresión **no aparece la variable aleatoria de la acción**: la acción está determinada por μ. Eso tiene dos consecuencias importantes. La primera es que el método es **off-policy por naturaleza**, sin necesitar muestreo por importancia: como no hay que muestrear acciones según π, cualquier política de *behaviour* sirve. La segunda es que requiere $\nabla_a q$, es decir, que la acción sea *continua*, y por eso DPG (Deterministic Policy Gradient) y su versión profunda DDPG (Deep Deterministic Policy Gradient) son los métodos de referencia en control continuo, donde el $\max_a$ de Q-learning sería un problema de optimización en sí mismo.

6. Visión unificada

Método	**Qué parametriza**	**Qué optimiza**	**Estimación del valor**
TD con aprox.	valor $\hat{v}(s,w)$	error cuadrático	—
Sarsa / Q-learning	valor $\hat{q}(s,a,w)$	error cuadrático	—
Deep Q-learning	valor (red neuronal)	error Bellman óptimo	—
REINFORCE	política $\pi(a\|s;\theta)$	$J(\theta)$	Monte Carlo
QAC	ambos	$J(\theta)$	TD sobre $\hat{q}$
A2C	ambos	$J(\theta)$	TD sobre $\hat{v}$ (advantage)
DPG	política determinista	$J(\theta)$	TD sobre $\hat{q}$

Las tres partes de esta guía son el mismo movimiento repetido. En la Parte I se parametriza el valor y se minimiza un error; en la Parte II se parametriza la política y se maximiza un rendimiento; en la Parte III se hacen las dos cosas a la vez, y la estimación del valor pasa a ser un ingrediente del cálculo del gradiente de la política.

Nota. Conviene retener qué se gana y qué se pierde al abandonar las tablas. Se gana poder trabajar con espacios de estados arbitrariamente grandes, y poder tratar acciones continuas, cosa imposible con un $\max_a$ tabular. Se pierden algunas garantías, como teoremas de convergencia. El punto donde esa pérdida se vuelve más visible es la combinación de aproximación, *bootstrapping* y aprendizaje off-policy.

7. Resumen

Concepto	**Expresión**
Objetivo del valor	$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2]$
SGD ideal	$w \leftarrow w + \alpha_w(v_\pi(s) - \hat{v}(s,w))\nabla_w \hat{v}(s,w)$
TD semigradiente	$w \leftarrow w + \alpha_w[r + \gamma\hat{v}(s',w) - \hat{v}(s,w)]\nabla_w \hat{v}(s,w)$
Caso lineal	$\hat{v}(s,w) = \phi^\mathsf{T}(s)w, \;\; \nabla_w \hat{v} = \phi(s)$
Caso tabular	$\phi(s) = e_s$
Deep Q-learning	dos redes (w, w_T) más repetición de experiencia
Softmax	$\pi(a\|s;\theta) \propto e^{h(s,a;\theta)}$
Truco del logaritmo	$\nabla_\theta \pi = \pi \, \nabla_\theta \ln \pi$
Gradiente de política	$\nabla_\theta J = \mathbb{E}[\nabla_\theta \ln \pi(A\|S;\theta) q_\pi(S,A)]$
Invarianza al baseline	restar $b(s)$ no cambia el gradiente
Ventaja	$A_\pi(s,a) = q_\pi(s,a) - v_\pi(s); \;\; \mathbb{E}[\delta_t] = A_\pi$
A2C	actor y crítico comparten el mismo δ_t

8. Ejercicios

Cada ejercicio incluye una *pista*. Las soluciones completas están en la Sección 9.

Ejercicio 1 (El caso tabular como caso lineal)**.** Sea $\phi(s) = e_s$ el s-ésimo vector canónico de $\mathbb{R}^{|\mathcal{S}|}$.

(a) Escriba $\hat{v}(s,w)$ y $\nabla_w \hat{v}(s,w)$ con esas características.

(b) Sustituya en (3) y compruebe que se recupera la actualización TD tabular.

(c) ¿Cuántas componentes de w cambian en cada paso? ¿Qué dice eso sobre la generalización?

Pista: $e_s^\mathsf{T} w$ selecciona una única componente de w.

Ejercicio 2 (Por qué *semi*gradiente)**.** Considere el objetivo $L(w) = \frac{1}{2}\big(r + \gamma\hat{v}(s',w) - \hat{v}(s,w)\big)^2$ para una transición fija.

(a) Calcule $\nabla_w L$ *correctamente*, derivando los dos términos que dependen de w.

(b) Compare con la dirección usada en (3) e identifique qué término se ha omitido.

(c) En el caso tabular con $\gamma = 0$, ¿coinciden ambas direcciones?

Pista: en (a) recuerde que $\hat{v}(s', w)$ también depende de w; su derivada aporta un término $\gamma \nabla_w \hat{v}(s', w)$.

Ejercicio 3 (Características y capacidad)**.** En la Sección 2.5 el RMSE pasó de 0.9278 con tres parámetros a 0.5135 con seis y a 0 con veinticinco.

(a) ¿Puede el RMSE aumentar al añadir características, manteniendo el ajuste por mínimos cuadrados?

(b) Si un agente aprende con (3) usando las tres características del plano, ¿puede alcanzar un RMSE menor que 0.9278?

(c) ¿Qué características añadiría para capturar que el valor es alto cerca de la casilla azul, sin pasar a 25 parámetros?

Pista: en (a), piense si la familia de funciones con seis características contiene a la de tres.

Ejercicio 4 (El gradiente de la softmax)**.** Sea $\pi(a|s;\theta) = e^{h_a} / \sum_{a'} e^{h_{a'}}$, con preferencias $h = (1,\ 0.5,\ 0,\ -0.5,\ 2)$.

(a) Calcule $\pi(\cdot|s)$.

(b) Demuestre que $\partial \ln \pi(a) / \partial h_{a'} = \mathbf{1}\{a' = a\} - \pi(a')$.

(c) Calcule $\nabla_h \ln \pi(a)$ para a la segunda acción y verifique que sus componentes suman cero.

(d) Explique por qué esa suma es necesariamente nula.

Pista: en (b), escriba $\ln \pi(a) = h_a - \ln \sum_{a'} e^{h_{a'}}$ y derive cada término.

Ejercicio 5 (Invarianza al baseline)**.** Con la política del ejercicio anterior y los valores $q = (3,\ -1,\ 0,\ 2,\ 5)$:

(a) Calcule $\mathbb{E}[\nabla_h \ln \pi(A)\, q(A)]$, promediando sobre $A \sim \pi$.

(b) Repita el cálculo con $q(A) - b$ donde $b = \mathbb{E}[q(A)]$.

(c) ¿Coinciden? ¿Qué cambia entonces al introducir el baseline?

Pista: para (a), el promedio es $\sum_a \pi(a)\, \nabla_h \ln \pi(a)\, q(a)$, un vector de cinco componentes.

Ejercicio 6 (El error TD es la ventaja)**.** Considere lo siguiente:

(a) Demuestre que $\mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) - v_\pi(S_t) | S_t = s, A_t = a] = A_\pi(s, a)$.

(b) ¿Por qué permite esto que el crítico de A2C estime sólo v en lugar de q?

(c) ¿Sigue siendo cierta la igualdad si en vez de v_π se usa una estimación $\hat{v}(\cdot, w)$ imperfecta?

Pista: para (a), reconozca el lado izquierdo como la definición de $q_\pi(s, a)$ una vez tomada la esperanza.

Ejercicio 7 (Las dos redes de DQN)**.** Considere lo siguiente:

(a) ¿Por qué el objetivo (7) es incómodo de derivar respecto de w?

(b) ¿Qué problema aparecería si la red objetivo se actualizara en cada iteración ($C = 1$)?

(c) ¿Y si no se actualizara nunca?

(d) ¿Podría usarse repetición de experiencia con Sarsa en lugar de Q-learning?

Pista: para (d), piense si el objetivo de Sarsa depende de la política que generó los datos.

Ejercicio 8 (Verdadero o falso)**.** Diga si cada afirmación es verdadera o falsa y justifique brevemente.

(a) Con aproximación de funciones siempre existe un w tal que $\hat{v}(s, w) = v_\pi(s)$ para todo s.

(b) El TD semigradiente es el descenso de gradiente de la función objetivo (1).

(c) REINFORCE y QAC se diferencian únicamente en cómo se estima $q_\pi(s_t, a_t)$.

(d) Restar un baseline reduce el sesgo del estimador del gradiente.

(e) El gradiente de política determinista no necesita muestreo por importancia para ser off-policy.

Pista: para (d), relea la Propiedad 5.1 y pregúntese si habla de sesgo o de varianza.

9. Soluciones

Solución 1.

(a) $\hat{v}(s, w) = e_s^\top w = w_s$, la s-ésima componente, y $\nabla_w \hat{v}(s, w) = e_s$.

(b) Sustituyendo en (3):

$$w_{t+1} = w_t + \alpha_w \big[r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t)\big] e_{s_t},$$

y como e_{s_t} tiene un único 1 en la posición s_t, esto equivale a

$$w_{t+1}(s_t) = w_t(s_t) + \alpha_w \big[r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t)\big],$$

dejando intactas las demás componentes. Escribiendo $v(s)$ en lugar de w_s, es exactamente TD(0) tabular.

(c) Cambia **una sola** componente por paso. No hay generalización alguna: lo aprendido en s_t no afecta a ningún otro estado. El caso tabular es el extremo de máxima capacidad (error nulo alcanzable) y mínima generalización.

Solución 2.

(a) Llamando $\delta = r + \gamma \hat{v}(s', w) - \hat{v}(s, w)$ y aplicando la regla de la cadena a los dos términos que contienen w:

$$\nabla_w L = \delta \left[\gamma \nabla_w \hat{v}(s', w) - \nabla_w \hat{v}(s, w)\right].$$

El descenso iría en la dirección $-\nabla_w L = \delta[\nabla_w \hat{v}(s, w) - \gamma \nabla_w \hat{v}(s', w)]$.

(b) La ecuación (3) usa la dirección $\delta\, \nabla_w \hat{v}(s, w)$: se ha omitido el término $-\gamma\, \delta\, \nabla_w \hat{v}(s', w)$, el que proviene de derivar el objetivo bootstrap. Por eso la actualización no es el gradiente de ninguna función, y se la llama semigradiente.

(c) Sí. Con $\gamma = 0$ el término omitido se anula, y ambas direcciones coinciden. Es coherente con que sin descuento no hay *bootstrapping*: el objetivo pasa a ser r, que no depende de w, y el problema se reduce a una regresión ordinaria.

Solución 3.

(a) **No.** Las seis características incluyen a las tres como subconjunto, de modo que cualquier función representable con el plano lo es también con la cuadrática (basta poner a cero los tres coeficientes nuevos). Como el ajuste por mínimos cuadrados elige el mejor elemento de la familia, ampliar la familia sólo puede mantener o reducir el error.

(b) **No.** El valor 0.9278 es el mínimo alcanzable *dentro de esa familia*, calculado con conocimiento perfecto de v_*. Un agente que aprende por TD a partir de muestras no puede superarlo; lo normal es que se quede algo por encima, y además el punto fijo de TD no coincide exactamente con el mínimo de (1).

(c) Cualquier característica que codifique proximidad al objetivo. Por ejemplo la distancia de Manhattan a la casilla 17, o una gaussiana centrada en ella, $\exp(-\|(x,y)-(3,2)\|^2/2\sigma^2)$. Con una o dos características de ese tipo se captura la estructura radial de v_*, que los polinomios de bajo grado aproximan mal. Es la idea que está detrás de las funciones de base radial.

Solución 4.

(a) Exponenciando y normalizando:

$$\pi = (0.2034,\ 0.1234,\ 0.0748,\ 0.0454,\ 0.5530),$$

que suma 1.

(b) Escribiendo $\ln \pi(a) = h_a - \ln \sum_{a'} e^{h_{a'}}$ y derivando respecto de $h_{a'}$: el primer término aporta $\mathbf{1}\{a' = a\}$, y el segundo

$$\frac{\partial}{\partial h_{a'}} \ln \sum_{a''} e^{h_{a''}} = \frac{e^{h_{a'}}}{\sum_{a''} e^{h_{a''}}} = \pi(a').$$

Restando se obtiene $\mathbf{1}\{a' = a\} - \pi(a')$.

(c) Con a la segunda acción, $\nabla_h \ln \pi(a) = e_2 - \pi$, es decir

$$(-0.2034,\ 0.8766,\ -0.0748,\ -0.0454,\ -0.5530),$$

cuya suma es $1 - 1 = 0$.

(d) Porque $\sum_{a'}(\mathbf{1}\{a' = a\} - \pi(a')) = 1 - \sum_{a'} \pi(a') = 1 - 1 = 0$. Geométricamente: la softmax obliga a que las probabilidades sumen 1, de modo que todo movimiento admisible de θ reparte probabilidad entre acciones sin crearla ni destruirla. Subir una exige bajar otras.

Solución 5.

(a) Con $\nabla_h \ln \pi(a) = e_a - \pi$, el promedio es

$$\sum_a \pi(a)\,(e_a - \pi)\,q(a) = \sum_a \pi(a) q(a) e_a - \Big(\sum_a \pi(a) q(a)\Big)\pi.$$

Numéricamente, $\sum_a \pi(a)q(a) = \mathbb{E}[q] = 3.3425$, y el vector resultante es

$$\begin{aligned}&(0.6103,\ -0.1234,\ 0,\ 0.0908,\ 2.7648) - 3.3425\,\pi \\ &\quad = (-0.0697,\ -0.5358,\ -0.2501,\ -0.0609,\ 0.9165).\end{aligned}$$

(b) Con $b = \mathbb{E}[q] = 3.3425$, el segundo término se anula porque $\sum_a \pi(a)(q(a) - b) = 0$, y queda

$$\sum_a \pi(a)(q(a) - b)e_a = (-0.0697,\ -0.5358,\ -0.2501,\ -0.0609,\ 0.9165).$$

(c) **Coinciden exactamente**, como garantiza la Propiedad 5.1: el baseline no altera el gradiente verdadero. Lo que cambia es la *varianza* del estimador construido con una sola muestra. Sin baseline, una muestra con $q = 5$ empuja fuertemente aunque esa acción sea sólo ligeramente mejor que la media; con baseline, el empuje es proporcional a $5 - 3.3425 = 1.6575$, que es la cantidad informativa. En el ejemplo, los valores centrados van de -4.34 a 1.66, en lugar de -1 a 5.

Solución 6.

(a) Por linealidad, condicionando en $S_t = s, A_t = a$:

$$\mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1})|s,a] - \mathbb{E}[v_\pi(S_t)|s,a] = q_\pi(s,a) - v_\pi(s) = A_\pi(s,a),$$

donde el primer término es exactamente la definición de $q_\pi(s,a)$ escrita con la ecuación de Bellman, y el segundo es $v_\pi(s)$ porque $v_\pi(S_t)$ está determinado al condicionar en $S_t = s$.

(b) Porque el actor sólo necesita la ventaja, y la ventaja se puede obtener del error TD, que a su vez sólo requiere $\hat{v}$. El crítico pasa de tener que aprender $|\mathcal{S}||\mathcal{A}|$ cantidades a aprender $|\mathcal{S}|$: menos parámetros y menos muestras.

(c) **No exactamente**. Con $\hat{v} \neq v_\pi$ el error TD deja de ser un estimador insesgado de la ventaja, y el actor recibe una señal sesgada. Es el precio habitual del *bootstrapping*: a cambio de menos varianza se acepta un sesgo que desaparece sólo cuando el crítico converge.

Solución 7.

(a) Porque w aparece dentro y fuera del máximo: tanto en $\hat{q}(S,A,w)$ como en $\max_a \hat{q}(S',a,w)$. Derivar el segundo término exige derivar a través de un máx y de la red evaluada en otro estado, lo que complica el cálculo sin aportar estabilidad.

(b) Con $C = 1$ desaparece la distinción entre las dos redes y el objetivo se mueve en cada paso al mismo ritmo que la estimación que intenta alcanzar. Es la situación clásica de «perseguir un blanco móvil»: el entrenamiento se vuelve inestable y puede oscilar o divergir.

(c) El algoritmo converge, pero a la solución equivocada: estaría resolviendo un problema de regresión contra un objetivo fijo $y = r + \gamma \max_a \hat{q}(s',a,w_T^0)$ que nunca incorpora lo aprendido. w_T debe actualizarse, sólo que despacio.

(d) **No, no de la misma manera.** El objetivo de Q-learning, $r + \gamma \max_a \hat{q}(s',a,\cdot)$, no menciona la política que generó los datos, de modo que transiciones antiguas siguen siendo válidas. El de Sarsa contiene $\hat{q}(s',a',\cdot)$ con a' elegida por la política *vigente en el momento de generar el dato*, que ya no es la actual. Reutilizar transiciones antiguas con Sarsa estima el valor de una política que el agente ya abandonó.

Solución 8.

(a) **Falsa.** Sólo si v_π pertenece a la familia representable. Con $m < |\mathcal{S}|$ eso es excepcional: en el ejemplo de la Sección 2.5 el mejor plano deja un RMSE de 0.9278. La excepción es el caso tabular, donde $m = |\mathcal{S}|$.

(b) **Falsa.** Omite el término $-\gamma \nabla_w \hat{v}(s',w)$ que resultaría de derivar el objetivo bootstrap, de modo que no es el gradiente de (1) ni de ninguna otra función. De ahí el prefijo *semi*, y de ahí que se pierdan las garantías de convergencia del descenso de gradiente.

(c) **Verdadera.** Ambos usan la actualización $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \ln \pi(a_t|s_t;\theta) q_t$. En REINFORCE, q_t es el retorno de Monte Carlo; en QAC, una estimación TD. De ahí se siguen las diferencias prácticas: REINFORCE es insesgado pero de alta varianza y necesita episodios completos; QAC aprende en línea a cambio de sesgo.

(d) **Falsa.** La Propiedad 5.1 dice que el gradiente *no cambia*, de modo que el estimador era ya insesgado y sigue siéndolo: el baseline no toca el sesgo. Lo que reduce es la **varianza**.

(e) **Verdadera.** Su expresión no contiene una esperanza sobre acciones muestreadas de π, porque la acción está determinada por $\mu(s;\theta)$. Al no haber que corregir ninguna distribución de acciones, cualquier política de *behaviour* sirve sin pesos de importancia.